Lab 4 - SIMD Processing Report - Sharique Baig 28369

1. Objective

The goal of this lab was to implement a 1D convolution kernel using AVX2 SIMD intrinsics and compare its performance against a standard scalar implementation. The focus was on understanding data parallelism, vector loading/storing, and fused multiply-add (FMA) operations.

2. Implementation Details

We implemented two versions of the 1D convolution algorithm:

2.1 Scalar Implementation (Baseline)

The scalar version processes one element at a time using standard C++ loops.

```
void convolve_1d_scalar(...) {
    for (int i = 0; i < output_size; i++) {
        float sum = 0.0f;
        for (int j = 0; j < kernel_size; j++) {
            sum += input[i + j] * kernel[j];
        }
        output[i] = sum;
    }
}
```

2.2 SIMD Implementation (Optimized)

The SIMD version uses AVX2 intrinsics to process 8 floating-point numbers simultaneously.

- **Loading**: `_mm256_loadu_ps` loads 8 consecutive input values.
- **Broadcasting**: `_mm256_set1_ps` broadcasts a single kernel value to all vector lanes.
- **Computation**: `_mm256_fmadd_ps` performs a fused multiply-add (`a * b + c`) in a single instruction.
- **Storing**: `_mm256_storeu_ps` writes the 8 results back to memory.

```
void convolve_1d_simd(...) {
    for (; i <= output_size - 8; i += 8) {
        __m256 sum = _mm256_setzero_ps();
        for (int j = 0; j < kernel_size; j++) {
            __m256 input_vec = _mm256_loadu_ps(&input[i + j]);
            __m256 kernel_vec = _mm256_set1_ps(kernel[j]);
            sum = _mm256_fmadd_ps(input_vec, kernel_vec, sum);
        }
        _mm256_storeu_ps(&output[i], sum);
    }
    // Tail loop handles remaining elements...
}
```

3. Verification Environment

- **Operating System**: Windows Subsystem for Linux (WSL) - Ubuntu 24.04
- **Compiler**: g++ 13.3.0
- **Flags**: `-O3 -mavx2 -mfma -fno-tree-vectorize`
  - `-fno-tree-vectorize` was used to prevent the compiler from automatically optimizing the scalar code, ensuring a fair comparison of manual SIMD implementation.

4. Results

The application was run with an input size of 1,000,000 elements and a kernel size of 5.

Output Evidence

```
=== 1D Convolution Challenge ===

Configuration:
  Input size:  1000000
  Kernel size: 5
  Output size: 999996
  Iterations:  1000

Input sample : [   0.50,    0.51,    0.52,    0.52,    0.53, ... ]
Kernel       : [   0.06,    0.24,    0.40,    0.24,    0.06]

=== Results ===
Correctness:
  Max error:    0.00
  Error count:  0 / 999996
  Status:       ✓ PASS

Performance:
  Scalar:    1048.87 ms
  SIMD:       454.36 ms
  Speedup:      2.31x
```

Analysis

- **Correctness**: The maximum error between scalar and SIMD output was 0.00, confirming that the SIMD implementation is functionally equivalent.
- **Performance**: The SIMD implementation achieved a **2.31x speedup** (454ms vs 1048ms). This demonstrates the significant advantage of vectorized processing for compute-intensive tasks like convolution.

5. Conclusion

The lab successfully demonstrated the power of SIMD programming. By manually leveraging AVX2 intrinsics, we more than doubled the performance of the convolution operation compared to the scalar baseline.